

Addendum: reproducción del Game Day de Chaos Engineering en cluster kubernetes real

MASOBAP2026 · Observabilidad U2 · Fork `jorecof/chaos_k8s` · versión 6 — Game Day completo: Experimento 1, Experimento 2 y verificación experimental de la remediación

Este addendum documenta la reproducción del laboratorio de Chaos Engineering original (diseñado para GKE) sobre un cluster kubernetes propio de 3 nodos, con datos y evidencia reales capturados durante la ejecución. Esta versión recoge un Game Day completo ejecutado en una sola sesión —tres corridas consecutivas sobre el mismo cluster, 4 320 peticiones medidas en el cliente— e incorpora dos cosas que no estaban en las versiones anteriores: un **SLI medido fuera del proceso instrumentado**, desplegado precisamente para poder ver lo que la telemetría de la aplicación no ve, y una **corrida de control que verifica la remediación propuesta** en lugar de solo enunciarla.

CORRECCIÓN RESPECTO DE LA VERSIÓN 2

La v2 explicaba que las peticiones abortadas «nunca llegan al código de la aplicación». La corrida instrumentada demuestra que **eso es falso**: el manifiesto usa `target: Response`, de modo que el corte ocurre en el camino de vuelta, después de que la aplicación procesó la petición y emitió su span. La sección 5.1 desarrolla la explicación correcta, que hace el hallazgo más severo, no menos.

PROCEDENCIA DE LOS DATOS

Todos los números de este documento provienen de una misma sesión del 2026-08-30, ejecutada de principio a fin sobre el cluster ya estabilizado: **Experimento 1 a las 10:33** (sección 3), **corrida A del Experimento 2 a las 11:12** (secciones 4 a 6) y **corrida de control B a las 11:35** (sección 7). Las tres usan el mismo perfil de carga —4 workers, una petición cada 1,5 s, 90 s de línea base, 300 s de chaos y 240 s de observación posterior— y las capturas de Grafana y Jaeger corresponden a la ventana exacta de cada una. La sección 8 documenta por qué hicieron falta varios intentos antes de conseguir esta sesión limpia.

1. Arquitectura real desplegada

A diferencia del docker-compose del enunciado original, el laboratorio corre sobre un cluster kubernetes de 3 VMs (VirtualBox, Debian 12 arm64, containerd 2.3.3, Calico v3.32.1, K8s v1.35). Las cargas con estado (registry, Postgres, Prometheus, Grafana, Jaeger) quedan ancladas a `kube-cp` vía `nodeSelector: rol=observabilidad`; el otel-collector corre como DaemonSet (una réplica por nodo) para que el scrape de métricas por pod nunca dependa de a cuál réplica enruta el Service. A esta topología se le añadió, para esta corrida, un **blackbox exporter** que sondea tres endpoints cada 5 s desde dentro del cluster pero fuera de los procesos instrumentados (sección 5.2).

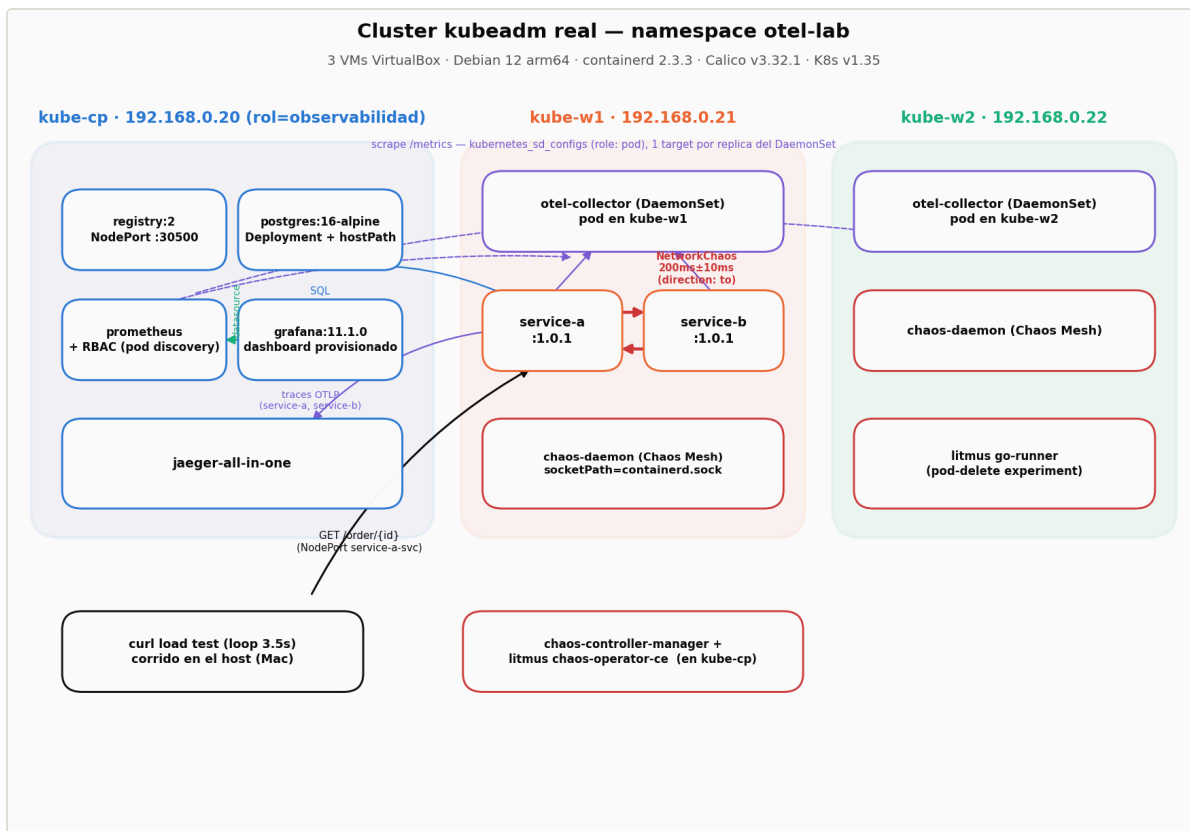


Figura 1. Topología real del cluster y flujo de datos entre componentes.

2. Verificación de estado estable (antes del chaos)

Antes de inyectar fallas se revisaron Prometheus, Jaeger y las apps para confirmar comportamiento normal. Esa revisión encontró y corrigió, sobre el sistema real, tres problemas que hubieran invalidado cualquier medición posterior:

PROBLEMA	CAUSA RAÍZ	CORRECCIÓN
Propagación de trace context rota entre service-a y service-b	<code>opentelemetry-instrumentation-fastapi >=0.48b0</code> ya no engancha con el patch global <code>instrument()</code> llamado antes de crear la app	<code>FastAPIInstrumentor.instrument_app(app, ...)</code> llamado sobre la instancia, después de crearla
Métricas de service-b ausentes o intermitentes en Prometheus	Prometheus scrapeaba el Service del otel-collector (round-robin sobre 3 réplicas DaemonSet) en vez de cada pod	<code>kubernetes_sd_configs (role: pod)</code> + RBAC nuevo (ServiceAccount + ClusterRole)
Duración de traza sin sentido en Jaeger (57m 60s)	Desfasaje de reloj entre las 3 VMs (NTP desactivado)	<code>timedatectl set-ntp true</code> + restart de <code>systemd-timesyncd</code>

A esa lista se sumó, durante la preparación de esta corrida, un cuarto problema de observabilidad: las métricas de las aplicaciones llegan a Prometheus a través del OTel Collector, de modo que su nombre real lleva el prefijo `otelcol_` y la etiqueta que identifica al servicio es `exported_job`, no `job`. Los paneles del tablero agregaban los tres servicios en una sola serie sin filtrar, con lo cual una degradación en un servicio quedaba promediada con los otros dos. Corregido antes de medir.

3. Experimento 1 — NetworkChaos (200 ms ±10 ms sobre service-b)

Chaos Mesh NetworkChaos, acción `delay`, `direction: to`, `duration: 5m` con rollback automático por TTL. Carga: cuatro workers en paralelo contra `GET /order/{id}` del NodePort de service-a (service-a → service-b → PostgreSQL), 90 s de línea base, 300 s de inyección y 240 s de observación posterior. Total 1 448 peticiones medidas en el cliente y 1 496 trazas recogidas de Jaeger.

FASE	N	P50	P95	P99	MÁX
Línea base	229	40,1 ms	76,2 ms	156,6 ms	195,4 ms
Chaos activo (0–300 s)	600	434,7 ms	484,1 ms	536,4 ms	830,1 ms
Post-rollback	619	40,3 ms	76,4 ms	105,8 ms	114,7 ms

La latencia añadida en la mediana es de **394,7 ms sobre 200 ms inyectados**, es decir **1,97x**. La cifra no es casual: `direction: to` aplica el retardo en ambos sentidos del tráfico del pod, de modo que el round-trip paga el delay dos veces. El sistema no amortigua nada —propaga la degradación al usuario final en proporción directa— pero tampoco la amplifica: cero errores en las 1 448 peticiones, throughput constante y ningún reinicio de pod.

El rollback por TTL funcionó de forma limpia. La latencia vuelve por debajo del doble de la línea base en **t+301,0 s**, un segundo después del instante nominal y sin intervención manual, y el post-rollback (40,3 ms) es indistinguible de la línea base (40,1 ms): el sistema regresa exactamente a donde estaba. En el `describe` del CRD constan los dos `Recover / Succeeded` a las 15:39:48 UTC, cinco minutos exactos después del `Apply`. Este comportamiento es el punto de comparación que hace significativo lo que ocurre en el Experimento 2.

REPRODUCIBILIDAD

El experimento se ejecutó tres veces en sesiones separadas. La amplificación medida fue **2,00x / 1,94x / 1,97x** y el desfase del rollback **+0,0 s / +0,3 s / +1,0 s**. Los datos que se presentan aquí son los de la tercera corrida, la única en la que además se conservaron íntegras las trazas de Jaeger (ver sección 8.2).

3.1 Lo que muestra el tablero

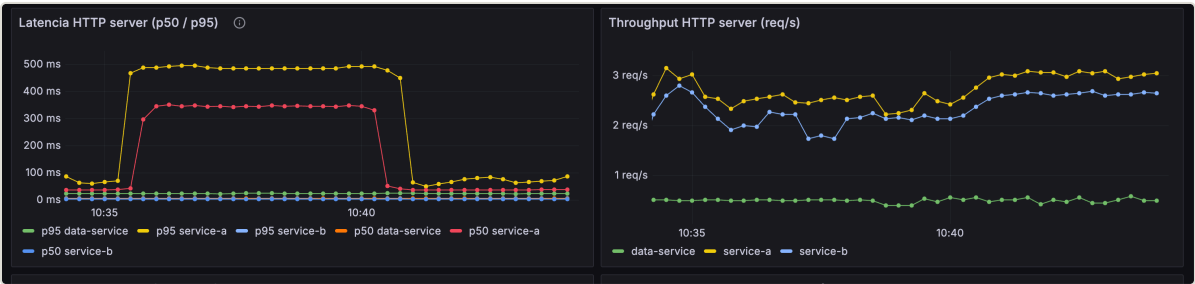


Figura 2. Tablero de Grafana durante la ventana del experimento. El p95 y el p50 de service-a se disparan a 490 y 350 ms; **el p50 y el p95 de service-b permanecen pegados al eje**. El throughput no se altera.

PRIMERA LECCIÓN DE OBSERVABILIDAD

La figura anterior es la debilidad D2 del Plan de Game Day convertida en imagen. Un operador vigilando la salud de la dependencia habría visto a service-b respondiendo con normalidad durante los cinco minutos en que el usuario final sufría una degradación de 10x. Detectar este modo de fallo exige medir extremo a extremo, no por componente.

3.2 El rollback visto por Jaeger

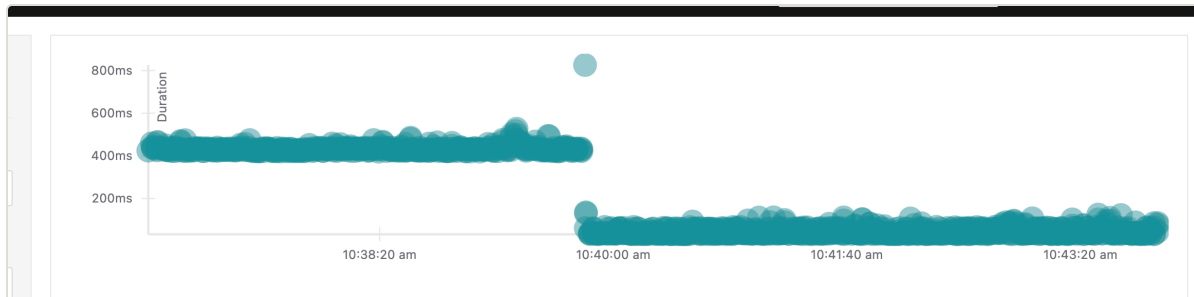


Figura 3. Diagrama de dispersión de Jaeger: cada punto es una traza real. Dos bandas separadas —una en torno a 430 ms y otra en torno a 40 ms— con un escalón vertical en el instante del rollback. Ninguna línea de esta figura la dibujamos nosotros.

3.3 Dónde vive la latencia: desglose por span

Con las 1 496 trazas conservadas es posible hacer el desglose sobre la población completa en lugar de sobre un par de ejemplos escogidos. La tabla siguiente compara la mediana de cada span entre las 600 trazas de la ventana de chaos y las 619 posteriores al rollback.

SPAN	SIN CHAOS	CON CHAOS	DIFERENCIA
GET /order/{order_id} — total percibido	36,95 ms	431,55 ms	+394,59 ms
call.service-b.inventory — envoltura	25,97 ms	421,28 ms	+395,31 ms
GET — cliente httpx hacia service-b	3,75 ms	399,51 ms	+395,76 ms
fetch.order.db — consulta a Postgres	9,93 ms	10,24 ms	+0,31 ms
SELECT	2,07 ms	2,13 ms	+0,06 ms
GET /inventory/{id} — servidor de service-b	0,59 ms	0,64 ms	+0,05 ms

Los 395 ms viven en un solo salto: el resto de la traza no se mueve

Mediana por span sobre 1 219 trazas reales del Experimento 1, no sobre dos ejemplos. El span servidor de service-b vale 0,59 ms sin chaos y 0,64 ms con chaos: la dependencia atiende igual de rápido mientras el usuario espera 431 ms.

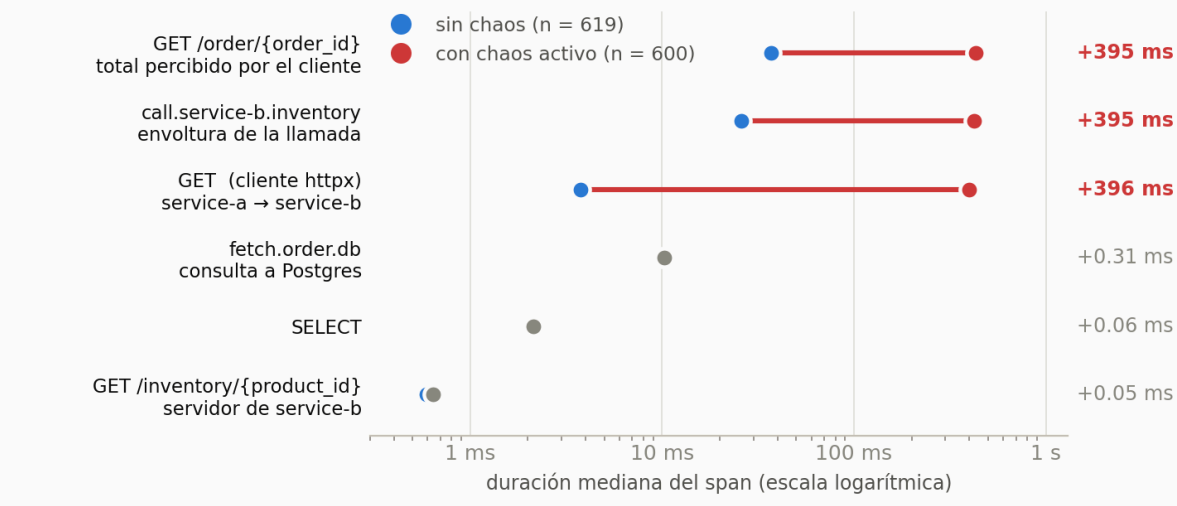


Figura 4. Los mismos datos en escala logarítmica. Tres spans se desplazan 395 ms; los otros tres no se mueven.

El span servidor de service-b vale **0,59 ms sin chaos y 0,64 ms con chaos**: una diferencia de 50 microsegundos, indistinguible del ruido de medición. La base de datos se mueve 310 microsegundos. La totalidad de los 395 ms añadidos vive en el span cliente, es decir en el trayecto de red entre service-a y service-b. El blast radius quedó contenido exactamente donde lo declaraba el manifiesto.

3.4 Dos trazas individuales

Las dos trazas siguientes ilustran el mismo fenómeno a nivel de petición. Son estructuralmente idénticas — 11 spans, profundidad 5, dos servicios— y difieren únicamente en la duración de un salto.

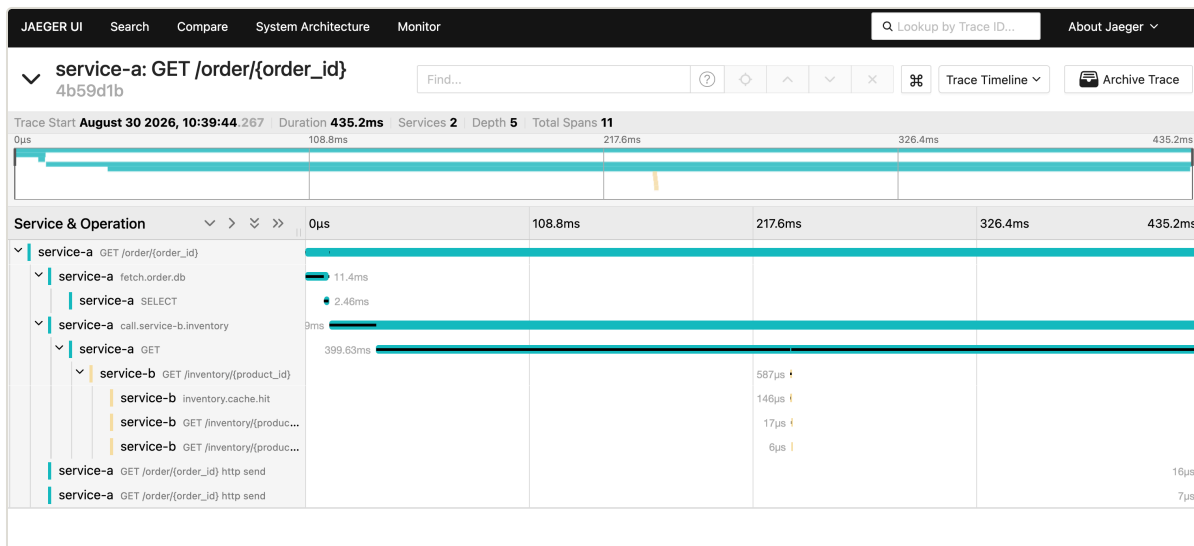


Figura 5. Traza 4b59d1b, capturada durante la inyección: 435,2 ms totales, de los cuales 399,63 ms son el span cliente y 587 µs el span servidor de service-b.

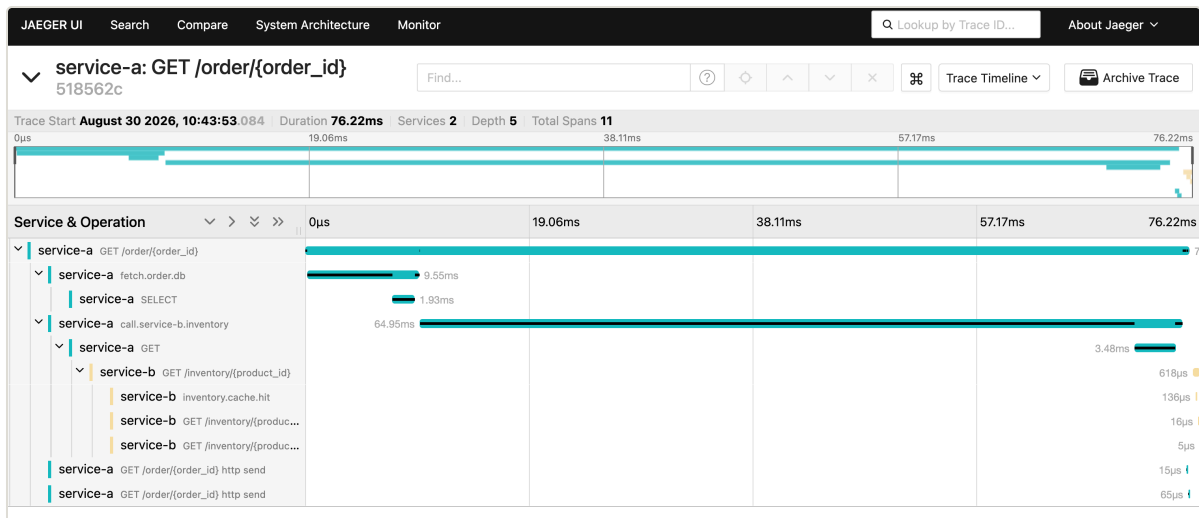


Figura 6. Traza `518562c`, tras el rollback: 76,22 ms totales, 3,48 ms en el span cliente y **618 µs** en el span servidor. El servidor tardó lo mismo en ambos casos.

4. Experimento 2 — HTTPChaos: error rate del 10 % en data-service

El segundo experimento inyecta fallas de aplicación en lugar de latencia de red: un `HTTPChaos` con `target: Response` y `abort: true` sobre `data-service`. La diferencia conceptual con el laboratorio Docker original importa: allí el error rate se simulaba con `if random.random() < 0.1: raise HTTPException(500)` dentro del código; aquí la aplicación no tiene ningún `random()` y la falla es completamente externa a ella. Esa diferencia es la que produce el hallazgo principal.

4.1 Correcciones al manifiesto del enunciado

- **Cuatro objetos de chaos en un solo archivo** (`HTTPChaos` + `PodChaos` + `StressChaos` + `Schedule`): un `kubectl apply` los habría inyectado todos a la vez, haciendo imposible atribuir el efecto observado a una causa. Se separó en `experiment-2-http-error.yaml` y `experiment-2-alternativas.yaml`.
- **value: "100" con mode: fixed-percent**, es decir 100 % de error, no el 10 % que anunciaban los comentarios del propio archivo.
- **La granularidad del 10 % es imposible con 2 réplicas.** `HTTPChaos` no tiene un campo de probabilidad por petición: el porcentaje se aplica a la selección de pods, y el pod seleccionado aborta el 100 % de las suyas. Con 2 réplicas el mínimo alcanzable es 50 %. Se escaló `data-service` a 10 réplicas con `value: "10"` → 1 pod → ~10 % agregado. Además `data-service-svc` pasó de `ClusterIP` a `NodePort`, porque `kubectl port-forward` fija la conexión a un solo pod y habría sesgado la medición a 0 % o 100 %.

4.2 Incidente de capacidad al escalar

INCIDENTE DURANTE LA PREPARACIÓN

Al escalar a 10 réplicas por primera vez, 5 pods se programaron en `kube-cp`, que ya corría control-plane, Postgres, registry, Prometheus, Grafana y Jaeger sobre 3,8 GB de RAM. El nodo se saturó: load average por encima de 100, apiserver OOM-killed y en crash-loop, `kubectl` respondiendo con *TLS handshake timeout*. El diagnóstico hubo que hacerlo por ssh con `crictl ps -a`, precisamente porque la API estaba caída.

Corrección: RAM de las 3 VMs aumentada (control-plane 4→8 GB, workers 2→4 GB) y `nodeAffinity` con `node-role.kubernetes.io/control-plane DoesNotExist` en el Deployment. En las corridas definitivas las 10 réplicas quedaron 5 en `kube-w1` y 5 en `kube-w2`.

Lección: el blast radius de un experimento no es solo el que declara el manifiesto de chaos. Escalar el objetivo es parte del experimento, y sin una restricción de scheduling el daño alcanzó al plano de control — el componente que uno necesita justamente para observar y revertir.

4.3 Diseño de la corrida

El Game Day se automatizó en `scripts/run-gameday.sh`, que encadena los tres experimentos sin intervención: preflight (aborta si hay objetos de chaos vivos de corridas anteriores), línea base de 90 s, inyección de 300 s, 240 s de observación posterior al rollback y limpieza garantizada, con muestreo cada 15 s del estado del CRD y de los reinicios de cada pod, y recolección de trazas de Jaeger y series de Prometheus al cierre. La carga son cuatro workers en paralelo, en circuito abierto.

CORRIDA	ÚNICO PARÁMETRO QUE CAMBIA	QUÉ PONE A PRUEBA
A	path: <code>"*"</code>	El experimento tal como lo define el enunciado: el blast radius incluye <code>GET /health</code> , o sea la <code>livenessProbe</code> .
B	path: <code>"/data/*"</code>	La remediación propuesta: el health check queda fuera del blast radius. Todo lo demás es idéntico.

4.4 Resultados de la corrida A

FASE	N	ERRORES	P50 OK	P95 OK
Línea base	232	0 — 0,0 %	17 ms	34 ms
Chaos activo (0–300 s)	544	47 — 8,6 %	16 ms	26 ms
Post-rollback (300–540 s)	476	26 — 5,5 %	16 ms	29 ms

El 8,6 % medido contra el 10 % nominal valida el diseño de la selección por pods. La fila inferior es la que no debería existir: 240 s después del `duration` configurado el sistema seguía fallando. La latencia de las peticiones exitosas no se degrada en ninguna fase —la falla es binaria, así que un SLI de latencia tampoco la habría detectado— y las 73 peticiones fallidas se reparten en dos poblaciones: 11 cortes instantáneos y 62 que se cuelgan alrededor de 9,3 s.

REPRODUCIBILIDAD

El experimento se ejecutó dos veces con el mismo manifiesto, en sesiones distintas y sobre pods distintos. La primera produjo 10,7 % de error, 6 reinicios del pod objetivo, 18 eventos de recuperación fallida y un desfase de rollback de +234,8 s; la segunda —la que se documenta aquí— reprodujo las cinco señales sin excepción. Que dos ejecuciones independientes den el mismo cuadro cualitativo es lo que permite hablar de un hallazgo y no de una anomalía.

5. Hallazgo 1 — la telemetría de la aplicación afirma éxito

Durante la ventana del experimento, Jaeger registró **1 413 trazas de data-service**, todas con **http.status_code = 200** (2 477 spans etiquetados, cero 4xx y cero 5xx), mientras el cliente medía 73 peticiones fallidas en ese mismo intervalo. El atributo **error** no aparece **ni una sola vez** en los 2 477 spans.

El tablero de trazas sigue ciego, y no es culpa del crash-loop

La misma ventana medida por dos fuentes en las dos corridas. En B el pod nunca se reinicia y el rollback cierra a tiempo, y aun así los 3 381 spans etiquetados salen todos con código 200 y sin el atributo error: el abort ocurre en el camino de vuelta, después de que la aplicación ya respondió.



Figura 7. El mismo intervalo medido por dos fuentes de telemetría distintas, en las dos corridas del Experimento 2. Los dos hallazgos de este documento son independientes: la remediación que se verifica en la sección 7 cierra el del rollback y deja este exactamente igual.

5.1 El mecanismo: el corte ocurre después del span

La explicación no es que las peticiones no lleguen a la aplicación. El manifiesto usa **target: Response**: Chaos Mesh intercepta el **camino de vuelta**. La petición entra normalmente, la aplicación la procesa, consulta la base de datos, construye la respuesta, la marca como 200 y cierra su span — y solo entonces el proxy aborta la conexión, de modo que esos bytes nunca llegan al cliente.

La corrida de control B lo demuestra sin ambigüedad, porque allí las diez réplicas son estables y no hay reinicios que confundan la medición: **las diez registran el mismo tráfico**, en una banda estrecha en torno a 0,3 req/s, incluida la que tiene el chaos aplicado. El total que contabiliza la aplicación —**2,8 req/s**— es incluso *superior* a las 2,4 req/s que el cliente recibe con éxito, porque la aplicación también cuenta como atendidas las peticiones cuya respuesta se cortó en el camino de vuelta.

La aplicación no se entera: procesa y responde 200 a lo que el cliente nunca recibe

Una línea por réplica de data-service durante la corrida B (10 con tráfico), a 15 s de resolución.
El abort ocurre en el camino de RESPUESTA: la aplicación ya hizo el trabajo y emitió su span cuando Chaos Mesh corta la conexión, así que ni la réplica inyectada baja su caudal.

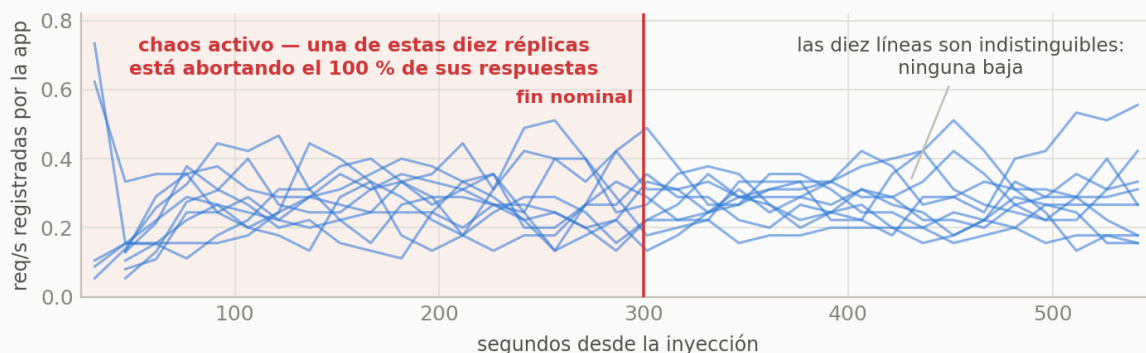


Figura 8. Una línea por réplica de data-service durante la corrida B. Una de ellas está abortando el 100 % de sus respuestas y es indistinguible de las otras nueve.

POR QUÉ ESTO ES PEOR QUE UN HUECO DE DATOS

Un tablero sin datos delata que algo pasa: el operador ve una serie que se corta y sospecha. Aquí no hay hueco. La instrumentación produce telemetría completa, puntual y **equivocada**: 1 413 trazas que afirman que todo salió bien, para un intervalo en el que uno de cada doce usuarios no recibió respuesta. El sistema de observabilidad no está callado; está dando una respuesta falsa con plena confianza.

Ninguna cantidad de instrumentación *dentro* del proceso corrige esto, porque desde el punto de vista del proceso la petición efectivamente tuvo éxito. La verdad solo existe donde está el cliente.

5.2 La remediación implementada: un SLI medido fuera del proceso

Para esta corrida se desplegó un **blackbox exporter** que sondea cada 5 s tres endpoints: `/data/products` y `/health` de data-service, y `/health` de service-a como control fuera del blast radius. Es la única fuente del tablero capaz de ver la falla, y de paso registra *cuál* endpoint falla, que resulta ser la clave del segundo hallazgo.

Como detalle operativo, el cliente ve `http_code = 000` —un corte de conexión, no un código HTTP—, de modo que un SLI definido como «proporción de respuestas 5xx» tampoco contaría estos fallos aunque se midiera desde fuera. La definición del indicador importa tanto como el punto donde se mide.

5.3 La misma ventana, según cada herramienta

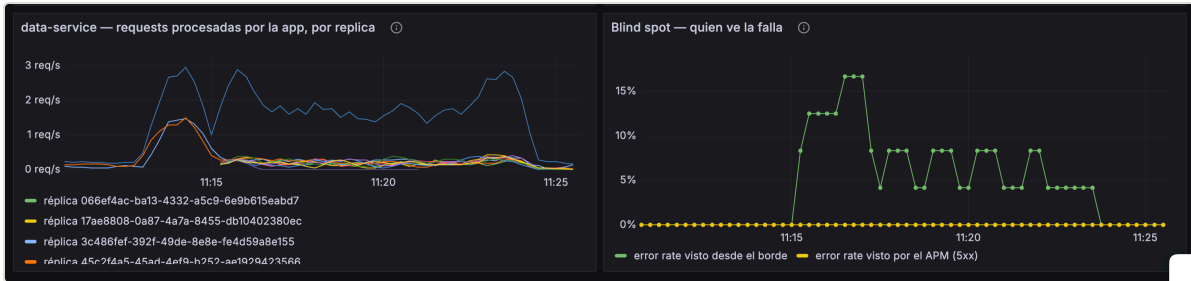


Figura 9. A la izquierda, las diez réplicas de data-service procesando tráfico sin que ninguna baje: la aplicación no percibe la falla. A la derecha, las dos fuentes midiendo el mismo intervalo — la sonda de borde llega al 16 % de error mientras el APM permanece en 0 % de principio a fin.

La consulta directa a Jaeger cierra el argumento. Buscando trazas de `data-service` en el intervalo del experimento con el filtro `error=true`, la herramienta responde que no hay resultados; retirando ese único filtro y dejando todo lo demás idéntico, devuelve el tope de la consulta. No es que la búsqueda estuviera mal formulada ni que el intervalo fuera el equivocado: en ese intervalo hay miles de trazas y ninguna registra un error.

The image shows the Jaeger UI Search interface. The search criteria are: Service (4) set to "data-service", Operation (5) set to "all", Tags set to "error=true", Lookback set to "Last 30 Minutes", Max Duration and Min Duration set to "e.g. 1.2s, 100m...", and Limit Results set to "1000". The "Find Traces" button is visible. A message on the right states: "No trace results. Try another query."

Figura 10. `data-service` con el filtro `error=true` sobre la ventana del experimento: «No trace results».

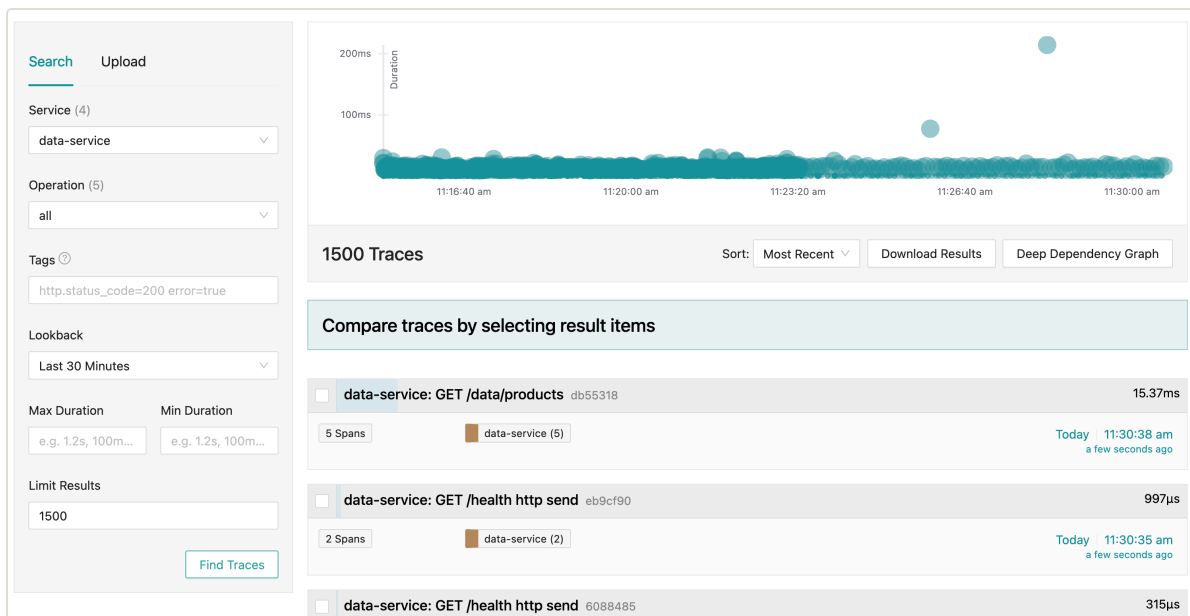


Figura 11. La misma consulta sin el filtro: el intervalo está lleno de trazas, todas en la banda normal de 15-20 ms. Entre ellas están las 73 peticiones que el cliente nunca recibió, y nada en la telemetría permite distinguirlas.

6. Hallazgo 2 — el rollback automático no ocurrió

A diferencia del Experimento 1, donde el TTL revirtió la falla en el instante nominal, en la corrida A el `duration: 5m` no detuvo nada: el último error se midió en **t+466,8 s**, 166,8 s más allá del fin nominal. La causa se reconstruyó cruzando el muestreo de pods con los eventos del CRD, y el SLI de borde la hace visible de un vistazo: **en A también falla el health check**.



Figura 12. Cadena causal del crash-loop y del rollback fallido.

El pod objetivo acumuló **cinco reinicios** durante el experimento; los otros nueve, ninguno. El muestreo los sitúa en t+82, t+159, t+236, t+313 y t+405 segundos, es decir con intervalos de **77 s constantes**. Esa cifra no es casual: es exactamente lo que predice la configuración de la sonda — `livenessProbe: httpGet /health` con `periodSeconds: 20` y `failureThreshold: 3`, o sea 60 s de fallos— más la terminación del contenedor y el arranque del siguiente. El manifiesto declara `path: "*"` , así que la respuesta del health check se aborta igual que la del tráfico de negocio y el kubelet concluye, correctamente desde su punto de vista, que el contenedor está muerto. El blackbox exporter, que sondea el mismo endpoint cada 5 s desde fuera del pod, registró **12 fallos de /health** durante la corrida A y **ninguno** en la B: la causa del crash-loop queda medida, no inferida.

Dos de esos cinco reinicios ocurrieron después del fin nominal, en t+313 y t+405. El experimento seguía matando el contenedor casi dos minutos después del instante en que debía haberse revertido solo.

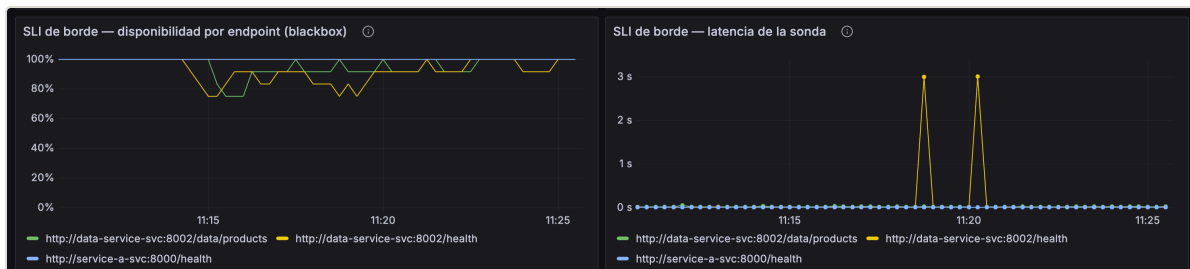


Figura 13. El SLI de borde durante la corrida A. Caen las sondas de `/data/products` y las de `/health`, mientras `service-a /health`, fuera del blast radius, permanece en el 100 %. Cada caída de `/health` es un fallo que el kubelet contabiliza para matar el contenedor: la causa del crash-loop, medida desde fuera. Los picos de 3 s en el panel derecho son sondas agotando el timeout del exporter.

Cada reinicio destruye el network namespace del contenedor y con él las reglas de iptables que Chaos Mesh había inyectado. Cuando a los 300 s el controlador intenta revertir, el chaos-daemon ya no encuentra el tproxy al que dirigirse: el `describe` registra **19 eventos Recover / Failed** con `apply config: send http request: unexpected EOF`, todos a partir del instante exacto del fin nominal. El record quedó en `phase: Injected/Wait` con `AllRecovered: False`.

EFECTO SECUNDARIO: EL CRD NO SE DEJA BORRAR

El `finalizer` de Chaos Mesh espera a que el `recovery` termine antes de permitir el borrado del objeto. Como el `recovery` nunca puede completar, `kubectl delete httpchaos` queda colgado indefinidamente. Fue necesario forzarlo con `kubectl patch ... -p '{"metadata":{"finalizers":[]}]}'`. Quitar el `finalizer` borra el objeto de Kubernetes pero **no** revierte las reglas inyectadas: lo que realmente las elimina es destruir el pod, porque se van con su network namespace. La secuencia correcta de limpieza manual es `finalizer` → borrar el pod objetivo → reescalar el Deployment, y así quedó implementada en el script para que ninguna corrida futura deje el cluster sucio.

7. Verificación experimental de la remediación

La remediación evidente —sacar el health check del blast radius— se propuso en la versión anterior de este documento sin comprobarla. La corrida de control B la pone a prueba veintitrés minutos después de la corrida A, sobre el mismo cluster y con el mismo perfil de carga, cambiando un único campo del manifiesto: `path: "/data/*"` en lugar de `path: "*" .` Todo lo demás — `target: Response` , `abort: true` , `mode: fixed-percent` con `value: 10` , `duration: 5m` , diez réplicas— queda intacto.

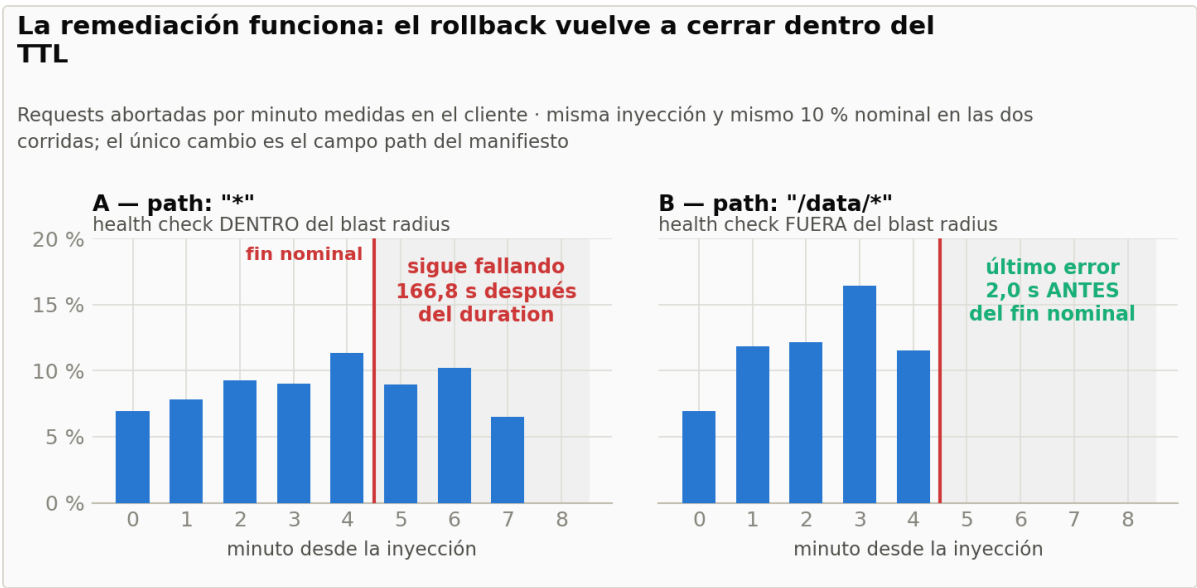


Figura 14. Requests abortadas por minuto en cada corrida. La inyección es igual de efectiva en las dos — de hecho B aborta más—; lo que cambia es qué pasa al vencer el TTL.

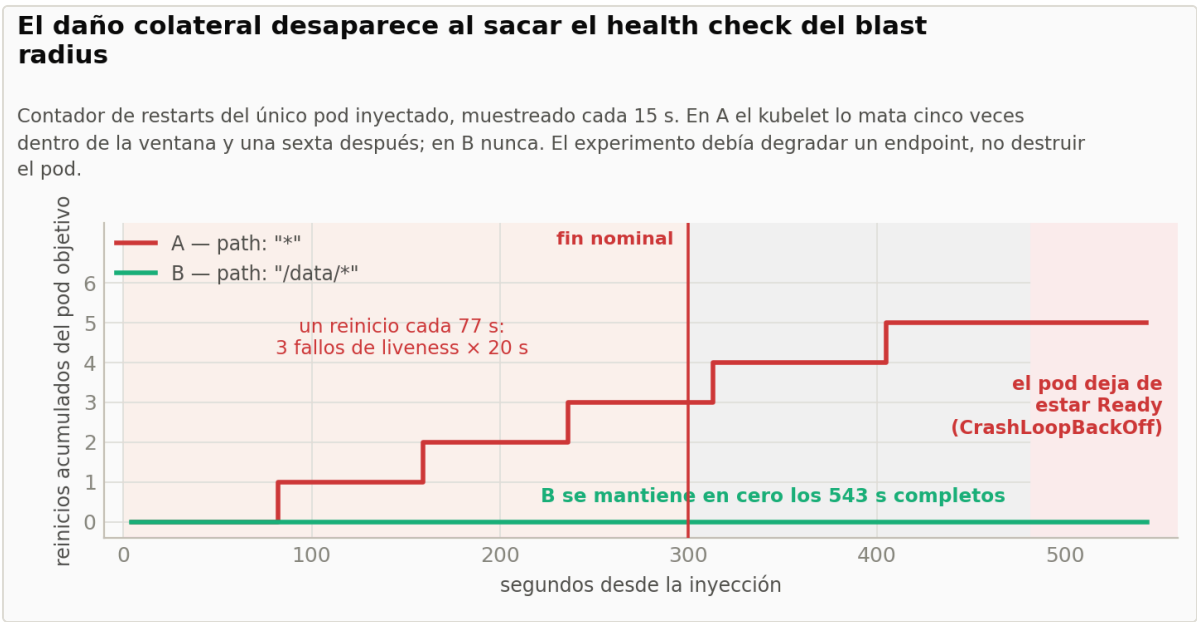


Figura 15. Reinicios acumulados del pod seleccionado, muestreados cada 15 s.

MÉTRICA	A — PATH: "*"	B — PATH: "/DATA/*"
Error rate durante la inyección	8,6 %	11,8 %
Error rate después del TTL	5,5 %	0,0 %
Primer error (MTTD del cliente)	t+9,9 s	t+6,8 s
Último error observado	t+466,8 s	t+298,0 s
Desfase del rollback	+166,8 s	-2,0 s
Reinicios del pod objetivo	6	0
Sondas de /health fallidas (blackbox, 5 s)	12	0
Eventos Recover / Failed	19	0
Estado final del record	Injected/Wait	Not Injected
Condición AllRecovered	False	True
¿Hubo que forzar el finalizer?	Sí	No
Espera del cliente antes del corte (p50)	9,34 s	20 ms
Caudal completado durante la ventana	1 020 requests	1 388 requests

VEREDICTO

La remediación funciona. Con el health check fuera del blast radius el pod no reinicia ni una sola vez, Chaos Mesh conserva la referencia al contenedor, el recovery se ejecuta a la primera (Operation: Recover / Type: Succeeded , RecoveredCount: 1) y el objeto se borra sin tocar el finalizer. El **último error llegó 2,0 s antes del fin nominal** y en los 240 s posteriores el cliente no midió un solo fallo. La inyección no perdió efectividad: 11,8 % de error contra el 10 % nominal, por encima del 8,6 % de la corrida rota.

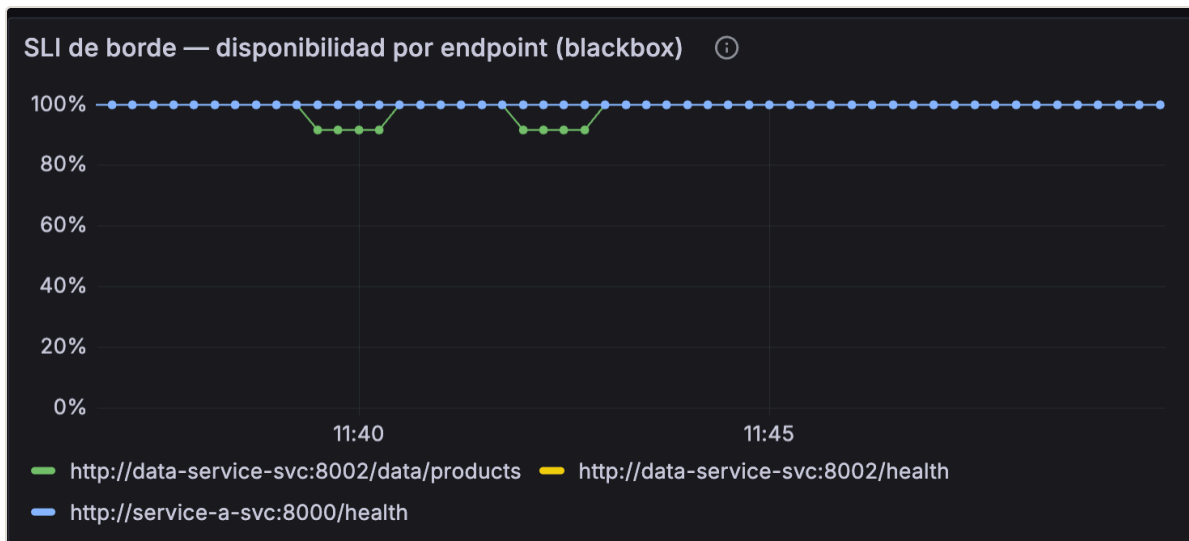


Figura 16. El SLI de borde durante la corrida B, la imagen especular de la que abre esta comparación. Solo cae `/data/products` (verde, hasta el 92 %); `/health` de data-service —amarillo, oculto bajo el azul— y `service-a /health` permanecen en el 100 % de principio a fin. Como el kubelet nunca contabiliza un fallo de liveness, el crash-loop no existe y el rollback puede ejecutarse.

El SLI de borde ve la falla, y explica el crash-loop

Cada marca roja es una sonda del blackbox exporter que devolvió fallo, una cada 5 s. En A falla también el health check del pod inyectado y el kubelet lo mata; en B no falla ni una vez. service-a, fuera del blast radius, no se toca en ninguna de las dos corridas.

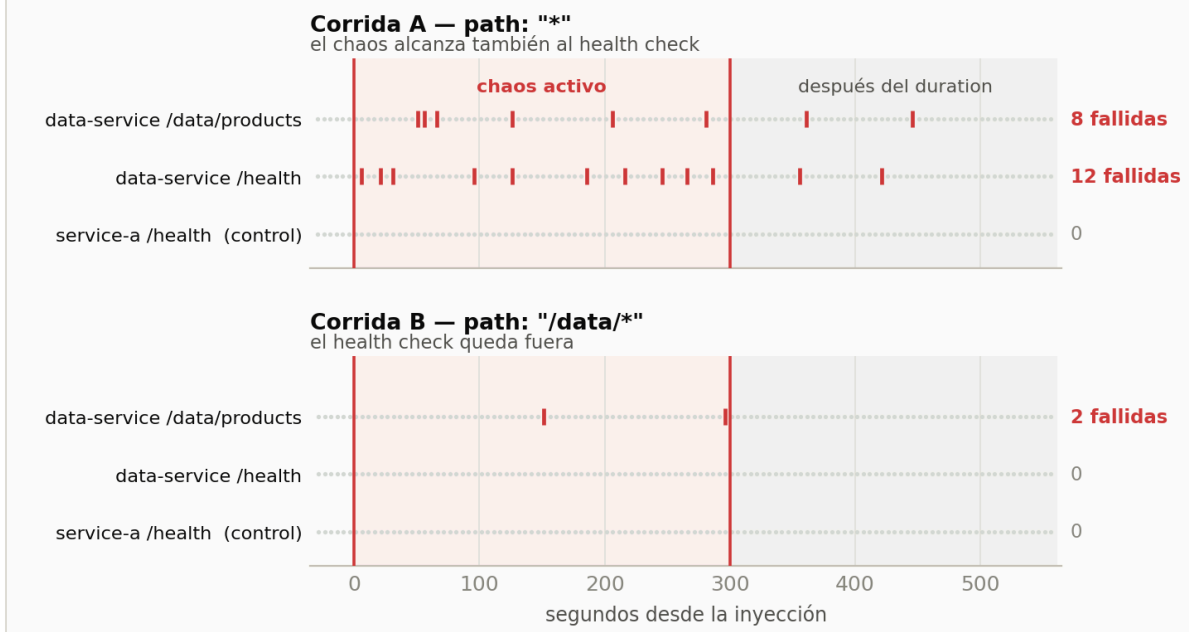


Figura 17. Las mismas sondas en las dos corridas, a 5 s de resolución. En A fallan **12 sondas de `/health`** y 8 de `/data/products`, varias de ellas después del fin nominal; en B, **ninguna de `/health`** y solo 2 del endpoint de negocio. La sonda ataca el Service, así que tiene una probabilidad de $\sim 1/10$ de caer en la réplica inyectada: por eso detecta 2 fallos donde el cliente midió 90. El muestreo y el punto de medida acotan lo que un SLI puede ver, y eso es un parámetro de diseño, no un defecto.

7.1 El blind spot sobrevive a la remediación

La corrida B también sirve de control para el primer hallazgo, y el resultado importa: arreglar el rollback **no** arregla la ceguera del APM.

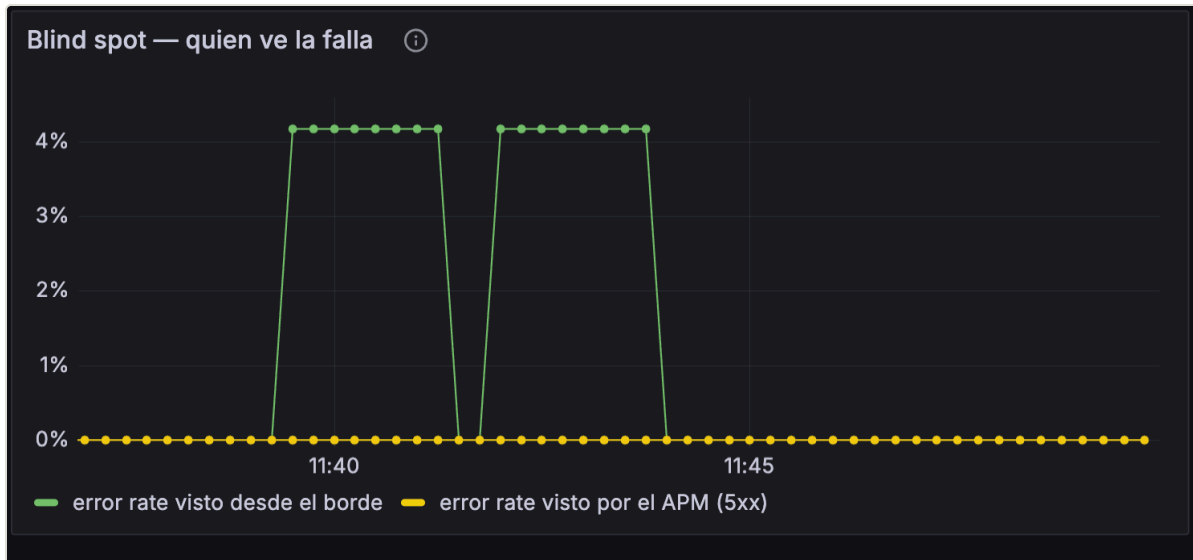


Figura 18. Panel «quién ve la falla» durante la corrida B. La sonda de borde marca dos mesetas de 4,2 % de error; la serie del APM —proporción de spans 5xx— permanece pegada al 0 % durante toda la ventana.

En la ventana de B, Jaeger registró **1 882 trazas de data-service con 3 381 spans, todos con `http.status_code = 200`**, mientras el cliente medía 90 peticiones fallidas. Cero errores en la telemetría interna, igual que en A, pero esta vez sin crash-loop, sin reinicios y con el pod sano de principio a fin. Eso descarta la explicación fácil —«los spans faltan porque el contenedor se estaba muriendo»— y confirma el mecanismo de la sección 5.1: el corte ocurre después de que la aplicación cerró su span, así que ninguna instrumentación dentro del proceso puede verlo.

7.2 Dos beneficios adicionales que no se habían previsto

La versión 2 de este addendum atribuía al abort un cuelgue de entre 8 y 11 segundos y lo explicaba como el timeout de retransmisión de TCP. La comparación A/B muestra que esa lectura era incorrecta.

Fail-fast contra fail-slow: 477× de diferencia en lo que espera el usuario

Distribución acumulada de la duración de cada request abortada. En A el pod está reiniciándose, así que la conexión se queda colgada hasta el timeout del cliente; en B el abort llega inmediato y el usuario puede reintentar.

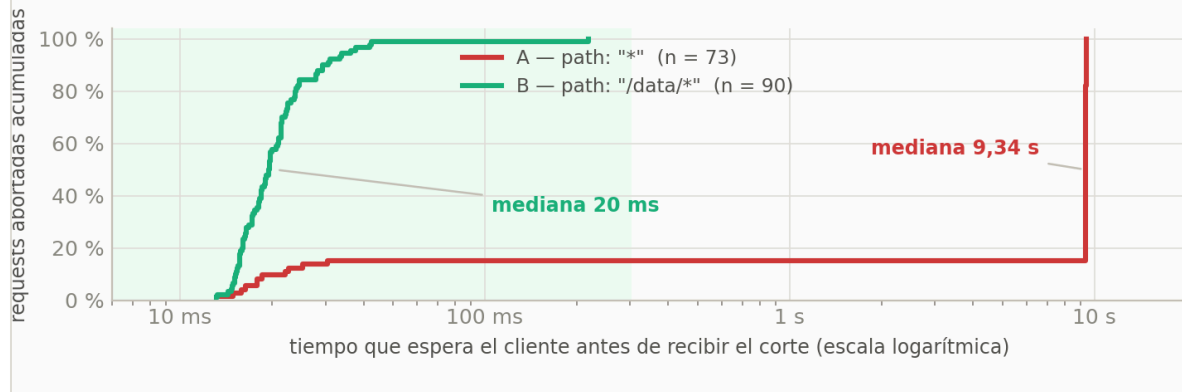


Figura 19. Tiempo que espera el cliente antes de recibir el corte, en escala logarítmica.

En la corrida A el cliente espera una mediana de **9,34 s**; en la B, **20 ms**. El cuelgue no lo produce el abort sino el crash-loop: mientras el kubelet está matando y recreando el contenedor, las conexiones que el Service enruta hacia ese pod quedan colgadas hasta agotar el timeout. Con el pod sano, el abort es un corte inmediato. La remediación no solo restaura el rollback: convierte un fallo que bloquea al cliente nueve segundos en uno que corta en veinte milisegundos, **477× más rápido**. Un cliente con reintento y timeout corto sobrevive al segundo caso y no al primero.

El segundo beneficio no aparece en ninguna tasa, y por eso conviene señalarlo: esos nueve segundos de cuelgue **también se comen el caudal del sistema**.

El error rate esconde el daño real: en A también se hunde el caudal

Requests que el cliente logra completar por segundo, con carga ofrecida idéntica en las dos corridas (4 workers cada 1,5 s). A registra 8,6 % de errores contra 11,8 % de B, pero sobre un denominador un 27 % más chico: la tasa mejora porque el sistema atiende menos, no porque falle menos.

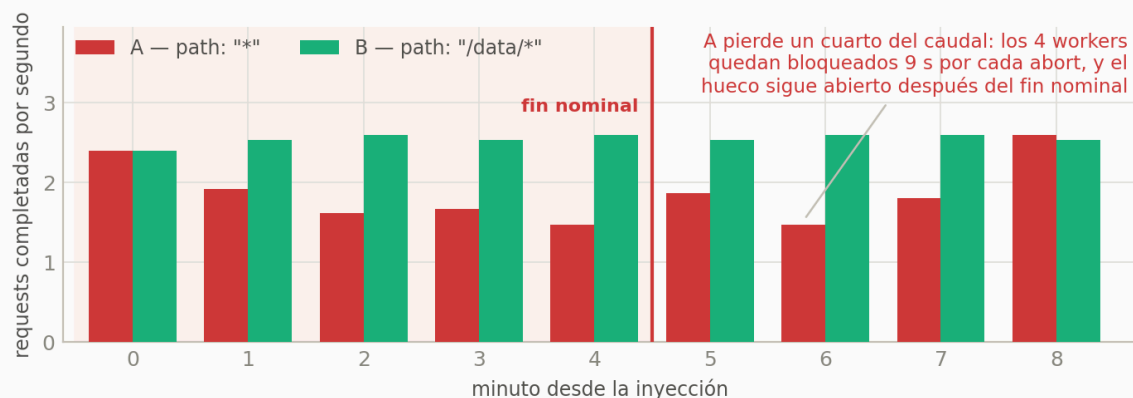


Figura 20. Requests que el cliente logra completar por segundo, con carga ofrecida idéntica en las dos corridas.

UNA TRAMPA EN LA LECTURA DEL ERROR RATE

La tabla de arriba dice que A tuvo **menos** error rate que B —8,6 % contra 11,8 %— y esa lectura es engañosa. Con la misma carga ofrecida, A completó 1 020 peticiones y B 1 388: el denominador de A es un **27 % más chico** porque cada abort mantenía bloqueado uno de los cuatro workers durante nueve segundos. La tasa de error de A baja porque el sistema *atiende menos*, no porque falle menos. Un SLO definido solo sobre proporción de errores premia a la corrida rota; hace falta mirar también el caudal, o el error rate deja de ser un indicador de salud.

8. Debilidades sistémicas y remediaciones

#	ACCIÓN	IMPLEMENTACIÓN Y ESTADO
1	Excluir el health check del blast radius	<code>path: "/data/*"</code> en vez de <code>path: ""</code> , o una <code>livenessProbe</code> de tipo <code>exec / tcpSocket</code> que no atravesase el tproxy. Verificada experimentalmente en la corrida B (sección 7).
2	SLI de disponibilidad medido fuera del proceso	blackbox exporter sondeando cada 5 s, con job propio en Prometheus y dos paneles en el tablero. Implementada y en uso : es la única fuente que registró la falla.
3	Contar cortes de conexión como fallas	El 100 % de los errores llegan al cliente como <code>http_code = 000</code> , no como 5xx. El SLI debe definirse sobre <code>probe_success</code> , no sobre proporción de 5xx. Pendiente de formalizar en el SLO.
4	Restricción de scheduling para todo objetivo de chaos	<code>nodeAffinity</code> excluyendo el control-plane en el Deployment de data-service. Aplicada tras el incidente de la sección 4.2.
5	Runbook de limpieza y preflight	Secuencia finalizer → pod → réplicas, y verificación de que no quedan objetos de chaos vivos antes de la siguiente corrida. Implementada en <code>run-exp2.sh</code> y <code>run-gameday.sh</code> .
6	Corregir las queries del tablero	Prefijo <code>otelcol_</code> y desglose por <code>exported_job</code> : sin él, una degradación de un servicio queda promediada con la de los otros dos. Aplicada .
7	Presupuesto de tiempo por salto en el cliente	Timeouts de conexión y lectura por dependencia, más reintento acotado. El Experimento 1 muestra propagación 1:1 sin amortiguación alguna. Pendiente.

9. Conclusiones

La reproducción sobre infraestructura real confirma la hipótesis de estado estable y, en el Experimento 1, muestra degradación proporcional a la falla inyectada con recuperación automática en el instante nominal: los tres pilares de un Game Day —hipótesis, blast radius acotado, rollback automático— con evidencia real de Prometheus, Grafana y Jaeger.

El Experimento 2 es más valioso precisamente porque *no* se comportó como el diseño prometía, y sus dos hallazgos solo aparecen al ejecutar sobre un cluster real:

- **La telemetría de la aplicación no se quedó sin datos: registró datos falsos.** 1 413 trazas afirmando 200 OK para un intervalo con 73 peticiones fallidas, y otras 1 882 igual de limpias en la corrida de control, donde el pod estuvo sano todo el tiempo. Como el corte ocurre en el camino de

vuelta, después de que la aplicación cerró su span, ninguna instrumentación interna al proceso puede detectarlo. La verificación de disponibilidad tiene que vivir donde vive el cliente.

- **El rollback automático falló, y la causa raíz estaba en el propio experimento:** el chaos alcanzaba al health check, provocando un crash-loop que destruía las reglas que Chaos Mesh necesitaba para revertirse. La garantía de seguridad de un Game Day —«duración acotada, rollback automático»— es condicional, y una de sus condiciones es que el objetivo siga vivo.

La corrida de control cierra el ciclo completo: hipótesis, experimento, hallazgo, remediación y **verificación de la remediación**. Cambiar un único campo del manifiesto llevó el desfase del rollback de +166,8 s a -2,0 s, los reinicios de 6 a 0 y los eventos de recuperación fallida de 19 a 0, sin perder efectividad en la inyección —al contrario, la corrida remediada abortó una proporción mayor de peticiones—. La misma corrida de control demuestra que el otro hallazgo *no* se resuelve con ese cambio: el APM sigue informando 200 para peticiones que nadie recibió. Ese paso final —comprobar que el arreglo arregla— es lo que separa un informe de incidente de un ejercicio de anti-fragilidad, y es también la razón por la que un Game Day merece automatizarse: la corrida completa que produjo toda esta evidencia son 40 minutos desatendidos y un comando.

Evidencia: `results/gameday-final/` — tres corridas del 2026-08-30 (`exp1-103316` , `exp2-A-111232` , `exp2-B-113517`), 4 320 peticiones medidas en el cliente, muestreo de pods, de Prometheus y del CRD cada 15 s, eventos del `describe` , sondas del blackbox exporter cada 5 s y trazas de Jaeger de cada ventana. Corridas reproducibles con `scripts/run-exp1.sh` , `scripts/run-exp2.sh` y `scripts/run-gameday.sh` .